ThreadKeep

Keep every thread accountable.

PRODUCT REQUIREMENTS DOCUMENT & PROJECT DEEP-DIVE

Project Type	Solo Product Build · MVP
Industry	Productivity / B2B SaaS
Build Period	April – May 2026
Stage	MVP Live · Validation Phase
Stack	n8n · Claude API · Google Sheets · Railway · GREEN-API
GitHub Repo	github.com/omshubham/ThreadKeep
Live Demo	omshubham.github.io/ThreadKeep
Document Version	1.0

Om Shubham
Product Manager · IIM Kozhikode · NIT Allahabad
omshubham99@gmail.com

1. Executive Summary

> *ThreadKeep is a passive AI agent that captures work signals across WhatsApp, Gmail, and Slack — turning fragmented conversations into a unified, searchable accountability layer for Indian professionals.*

In Indian workplaces, personal WhatsApp drives **80%+ of real work coordination**, yet no enterprise tool monitors it. Existing productivity solutions like Asana, Slack AI, and Notion operate within their own platform silos and require manual task creation, while WhatsApp Business API serves customer-facing use cases, not internal team accountability.

The opportunity: a cross-platform intelligence layer that captures commitments **without changing user behavior** — addressing a pain point with high willingness-to-pay (the "*maine kab bola?*" escalation moment) that no incumbent has solved.

2. Problem Statement

Modern Indian workplaces operate across fragmented communication channels — personal WhatsApp groups, Gmail threads, and Slack channels are used simultaneously and often interchangeably for work coordination. As conversations span multiple platforms, critical tasks, decisions, and commitments get scattered and buried under message volume, with no unified system of record.

The consequence is **accountability erosion**: when deliverables slip or commitments are disputed, there is no reliable way to retrieve who agreed to what, when, or in which channel.

Validated Pain Layers

Layer	What Users Experience	Severity
Task Vanishing	Tasks assigned in chats disappear under hundreds of messages within hours	Daily
Decision Burial	15+ minutes weekly searching for past decisions across platforms	Weekly
Accountability Collapse	"Maine kab bola?" — career-affecting escalation moment	Monthly · High WTP

Source: 15 user interviews with corporate professionals (24–35) across Bangalore, Mumbai & Delhi NCR.

3. Challenge & Constraints

· **Zero-API constraint** Personal WhatsApp has no official API. Competitors have repeatedly failed to crack this — the technical wall most enterprise tools refuse to engage with.

· **Multi-channel parity** The solution must extract signals consistently across structured (Slack), semi-structured (Gmail), and unstructured (WhatsApp) inputs — without losing nuance.

· **Hinglish nuance** Indian work communication blends English and Hindi (*"kal tak deck bhej dena"*). Rule-based parsing fails here; the system needs contextual NLP.

· **Trust threshold** Users will not forward private chats to a third party without absolute clarity on data handling, retention, and access.

· **Bootstrap economics** Built solo with zero engineering team. Required full reliance on no-code orchestration, with every decision optimised for shipping speed.

4. Solution Overview

ThreadKeep follows a **four-layer event-driven pipeline**. Each layer is independently replaceable — a deliberate design choice to avoid lock-in as the product scales.

1. INGESTION	Gmail OAuth × 2 · Slack Webhook · WhatsApp via GREEN-API
2. ORCHESTRATION	n8n self-hosted on Railway — routes & formats messages with source tags
3. EXTRACTION	Claude API (claude-sonnet-4-6) — extracts structured signals from raw text
4. SURFACE	Google Sheets database → live HTML dashboard + WhatsApp SimSim command

Four-layer architecture — each layer independently replaceable.

Six Signal Types Extracted

TASK	Someone is asked to do something	"Send the deck by EOD"
DECISION	A choice has been finalised	"We are going with vendor B"
COMMITMENT	Someone has volunteered	"Main le leta hoon"
BLOCKER	Work is actively stuck	"Cannot proceed without API access"
MEETING	A sync has been scheduled	"Let us connect at 5 PM tomorrow"
FYI	Information of relevance	"FYI the client moved the deadline"

4.1 Feature Inventory

A complete list of features — both shipped in MVP and explicitly deferred. The **green checks** mark what is live in production today; **amber dots** mark features designed but not yet built.

Status	Feature	Description	Phase
● LIVE	**Gmail OAuth Ingestion**	Auto-scans incoming emails from 2 accounts	MVP
● LIVE	**Slack Bot Listener**	Real-time message capture from channels	MVP
● LIVE	**WhatsApp Forward Ingestion**	GREEN-API webhook receives forwarded messages	MVP
● LIVE	**AI Signal Extraction**	Claude API extracts 6 signal types with structured JSON	MVP
● LIVE	**Hinglish Understanding**	Parses code-switched English+Hindi natively	MVP
● LIVE	**Google Sheets Storage**	Persistent signal database with 11 columns	MVP
● LIVE	**Live Dashboard**	HTML dashboard with KPIs, filters, search	MVP
● LIVE	**Status Sync (Open/Closed)**	Two-way sync between dashboard & Sheets	MVP
● LIVE	**Mark All Done + Undo**	Bulk close action with 10-second undo banner	MVP
● LIVE	**Dark / Light Mode**	Theme toggle with persisted preference	MVP
● LIVE	**Privacy Mode**	Blur all signal content with one click	MVP
● LIVE	**SimSim WhatsApp Command**	Secret keyword triggers status digest reply	MVP
● LIVE	**24/7 Cloud Deployment**	Railway-hosted n8n with Postgres	MVP
■ PLANNED	**Voice Note Transcription**	Whisper-based transcription before extraction	Phase 1
■ PLANNED	**Promotional Email Filter**	Skip CATEGORY_PROMOTIONS / SPAM upstream	Phase 1
■ PLANNED	**Smart Reminders**	WhatsApp/Slack DM 24h before / 1h after deadline	Phase 1
■ PLANNED	**Weekly Digest**	Monday 9 AM email summary of past week	Phase 1
■ PLANNED	**Escalation Snapshot**	"What did Rahul commit to?" structured query	Phase 1
■ PLANNED	**Duplicate Detection**	Skip rows with same summary + sender + source	Phase 1
■ PLANNED	**Chat Export Parser**	Bulk-process WhatsApp .txt exports for backfill	Phase 2
■ PLANNED	**Image / Screenshot OCR**	Extract signals from forwarded screenshots	Phase 2
■ PLANNED	**Multi-Tenant Architecture**	Per-user data isolation in Postgres	Phase 2
■ PLANNED	**PII Redaction Layer**	Strip account numbers, OTPs before Claude call	Phase 2
■ PLANNED	**Team Dashboard (Linear)**	Shared view across team members	Phase 3
■ PLANNED	**Manager Digest**	Aggregated view of team commitments	Phase 3
■ PLANNED	**Conflict Detector**	Flag overlapping deadlines for same person	Phase 3
■ PLANNED	**Self-Hosted / SSO / Audit**	Enterprise compliance bundle	Phase 4

13 features live in MVP · 14 planned for Phases 1–4

4.2 Prioritization Rationale

Why these 13 features made the MVP cut while 14 others were deferred. Each MVP feature was scored against four criteria: **core loop dependency**, **user trust impact**, **build effort**, and **differentiation**. Features that scored high on the first two and low on the third were always shipped first.

Feature (Shipped)	Why Prioritized	Trade-off Accepted
Gmail + Slack + WhatsApp ingestion	Core loop fails without all 3 channels. Gmail+Slack alone = same as competitors.	Took longer than building Gmail-only MVP would have
AI Signal Extraction (Claude)	The product's entire value proposition. Without it, this is just a forwarding inbox.	API costs scale with usage — accepted as an MVP cost
Hinglish Understanding	Non-negotiable for Indian market. English-only would fail the core user persona.	No specific tuning — relied on Claude's base capability
Google Sheets Storage	Fastest path to persistent storage. Visible to user (transparent debug surface).	Doesn't scale beyond ~10K rows. Migration plan defined upfront
Live Dashboard	Without a UI, users can't verify signals are captured correctly. Trust killer.	HTML-only, no responsive mobile yet
Status Sync (Open/Closed)	Tasks are useless if you can't mark them done. Persistent state = real productivity tool.	Required building 2 webhook endpoints + UUID system
SimSim WhatsApp Command	Killer differentiator. Lets users get status without opening dashboard — the "magic moment".	Hardcoded to one user for MVP — multi-user deferred to Phase 2
24/7 Cloud Deployment	A laptop-only product is a demo, not an MVP. Must work when user is asleep.	Railway free tier expires in 30 days — accepted near-term cost

Why These Features Were Deferred

Deferred Feature	Why Not in MVP
Voice Note Transcription	Adds API cost & latency. MVP validated extraction works on text first — voice is incremental, not foundational.
Smart Reminders	Requires reliable deadline parsing accuracy. Decided to ship extraction first, validate quality, then add reminders.
Weekly Digest & Escalation Snapshot	Solved by SimSim command for MVP — both can be built later as scheduled triggers without changing core architecture.
Multi-Tenant Architecture	Single user for validation phase. Multi-tenancy is a re-architecture effort, not a feature — best added once usage patterns are understood.
PII Redaction	Critical for production but not for solo testing. Prematurely building it would slow MVP without protecting any real users yet.
Team Features	Phase 3 by design. Solo product-market fit must be proven before adding multiplayer complexity.
Image/Screenshot OCR	Edge case. Most forwarded WhatsApp messages are text. OCR doubles complexity for <15% of inputs.
Conflict Detector	Requires a critical mass of signals to be useful. With 12 signals, conflicts are obvious to the eye.

5. Example Flow — End-to-End Walkthrough

Below is a concrete trace of one signal moving through ThreadKeep, from raw WhatsApp message to dashboard entry to SimSim reply. This is the actual flow that runs in production.

STEP 1 — Original message arrives in a WhatsApp group

> **Rahul** · Project Alpha group · 9:14 AM
>
> *"Bhai kal tak deck bhej dena client ke liye. Urgent hai."*

STEP 2 — User forwards to ThreadKeep number (single tap)

Long-press the message → Forward → ThreadKeep WhatsApp number. Total friction: ~2 seconds.

STEP 3 — GREEN-API delivers webhook payload to n8n

```
{
  "typeWebhook": "incomingMessageReceived",
  "senderData": { "senderName": "Shubham", "chatId": "917607193685@c.us" },
  "messageData": { "textMessage": "Bhai kal tak deck bhej dena client ke liye. Urgent
hai." }
}
```

STEP 4 — n8n formats and forwards to Claude API

The orchestration node prepends a source tag and routes to the Claude LLM with a system prompt that defines the six signal types and Hinglish handling rules.

STEP 5 — Claude returns structured JSON

```
{
  "signals": [{
    "type": "TASK",
    "summary": "Send client deck",
    "person": "message recipient",
    "deadline": "tomorrow",
    "priority": "high",
    "confidence": "high"
  }],
  "source": "whatsapp",
  "original_sender": "Rahul"
}
```

STEP 6 — Row appended to Google Sheets with UUID

Type	Summary	Person	Deadline	Priority	Source	Status
TASK	Send client deck	recipient	tomorrow	high	whatsapp	open

Plus: ID (UUID) · Confidence · Sender · Timestamp columns

STEP 7 — Dashboard auto-refreshes with new signal

The dashboard polls the n8n webhook every 2 minutes. Within 2 minutes, the new TASK card appears at the top of the list, color-coded blue, with the WhatsApp source badge.

STEP 8 — Later, user sends "SimSim" to ThreadKeep number

You · 7:03 PM

SimSim

STEP 9 — ThreadKeep replies with structured status digest

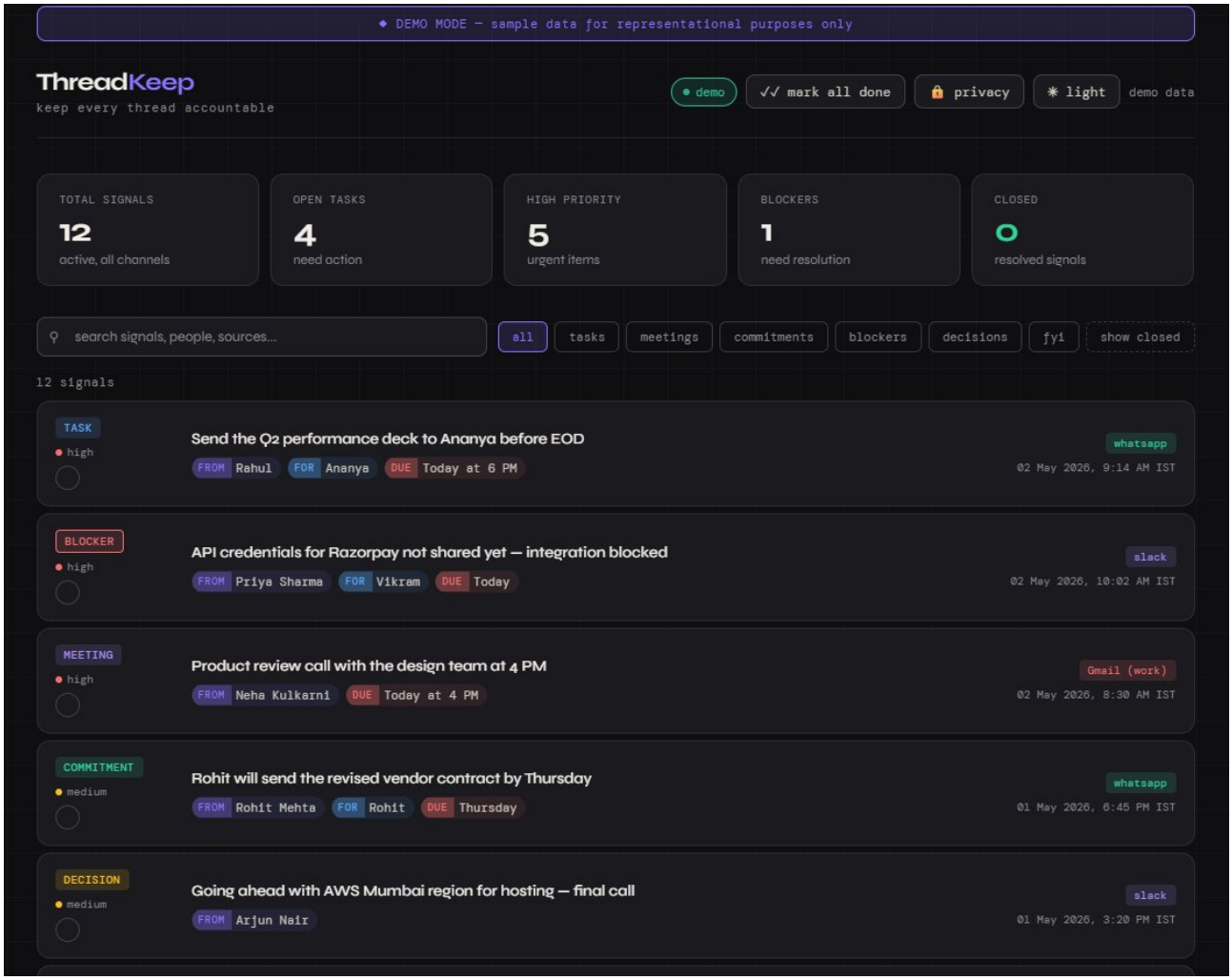
```
ThreadKeep Status · 02 May, 7:03 PM IST
12 open signals total

Recent 5:
1. [TASK] Send client deck → recipient | tomorrow
2. [BLOCKER] Razorpay credentials missing → Vikram | today
3. [MEETING] Product review at 4 PM
4. [COMMITMENT] Rohit will send vendor contract | Thursday
5. [DECISION] AWS Mumbai region — final

Top Priority:
1. [TASK] Marketing budget approval | tomorrow
2. [TASK] Upload signed NDA | tomorrow

+ 5 more signals. View full dashboard.

ThreadKeep · your accountability agent
```

Total round-trip: ~3 seconds from "SimSim" to digest delivery.

6. Product Walkthrough — Dashboard

Dark Mode (default)

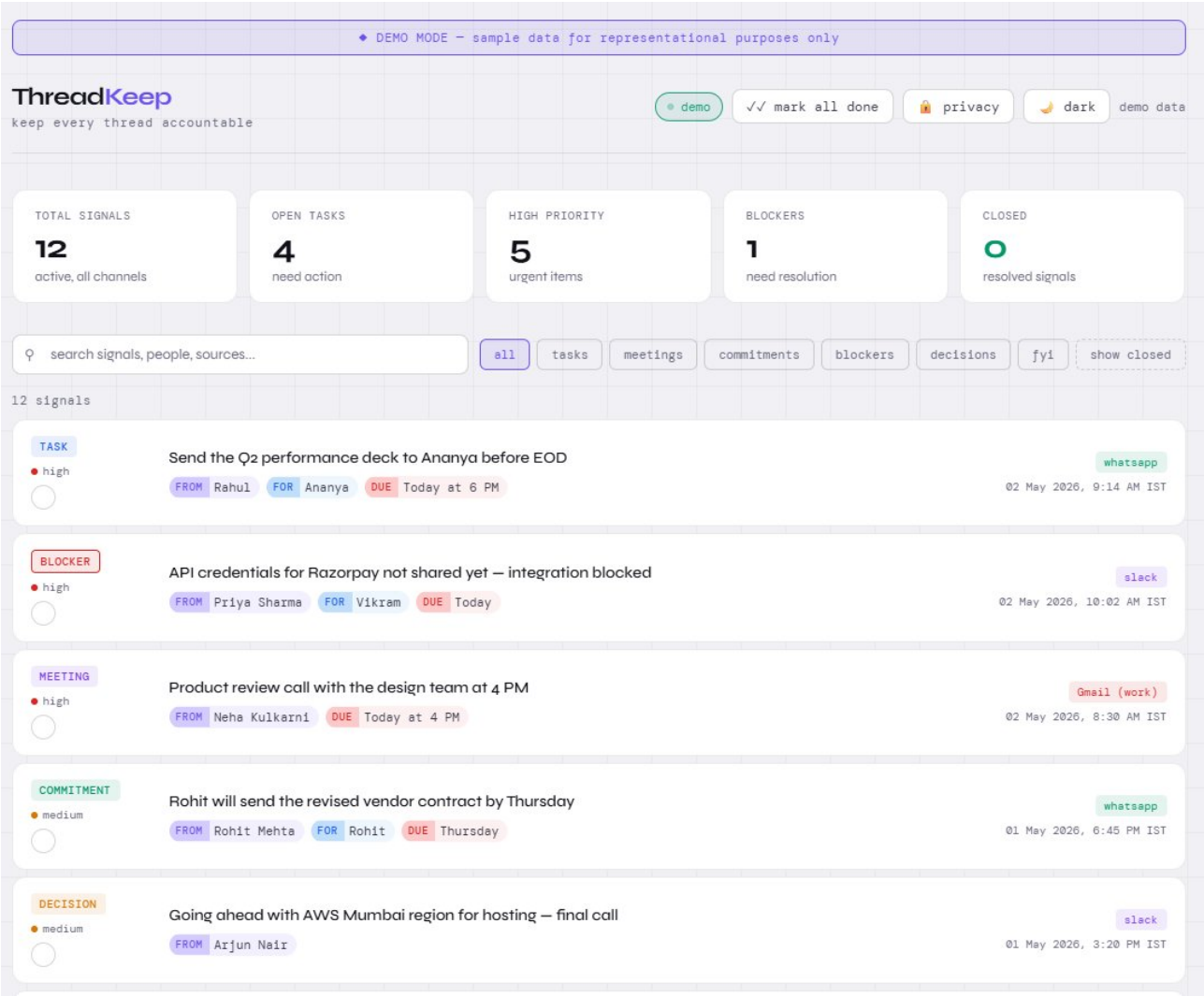
The dashboard shows KPI cards at the top — **Total Signals · Open Tasks · High Priority · Blockers · Closed** — followed by a filterable, searchable list of signals. Each card displays type, priority, source, sender, assignee, deadline, and timestamp. Dark mode is the default theme, optimised for long viewing sessions.



Dark mode dashboard — default view with 12 sample signals.

Light Mode

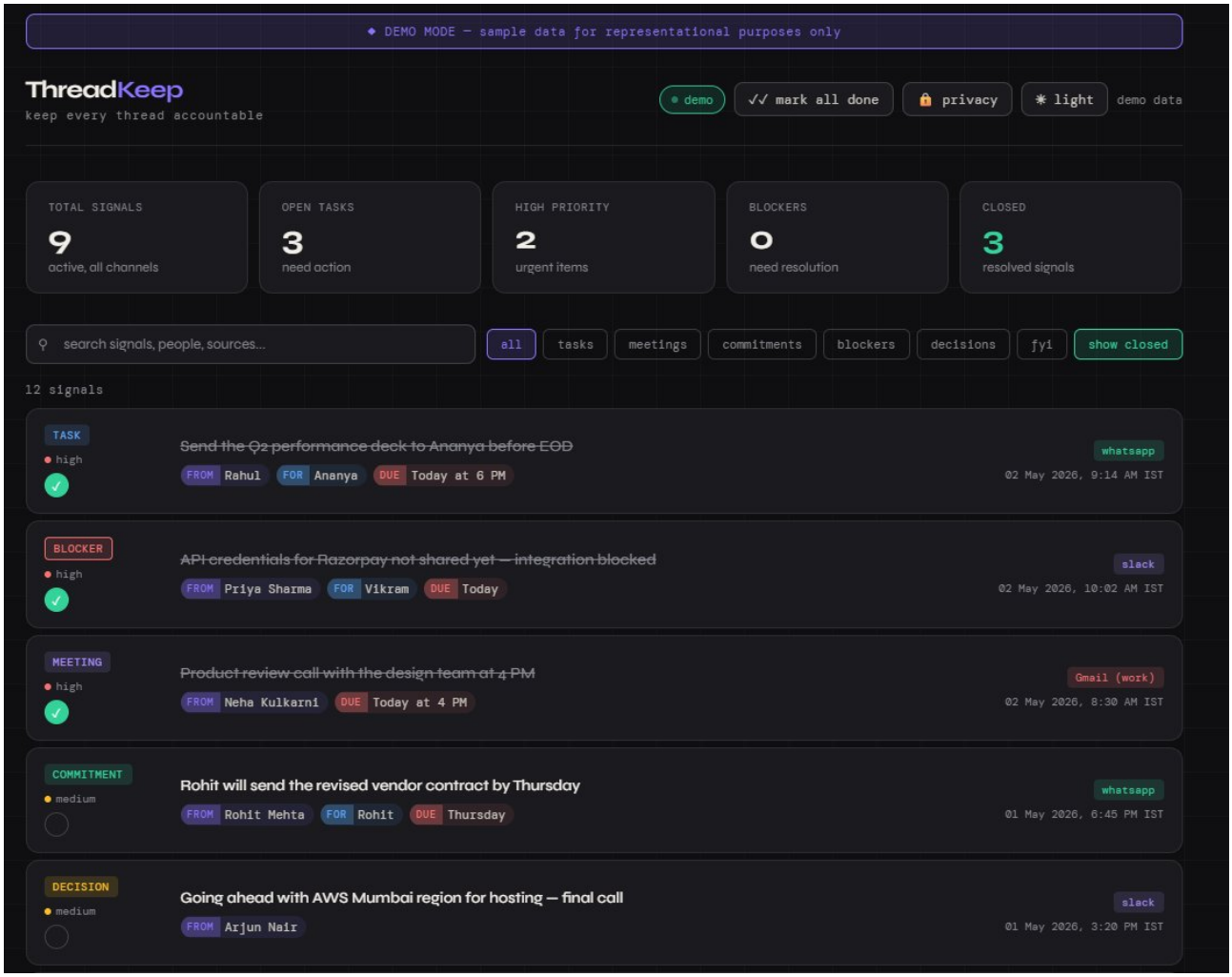
A toggle switches to light mode for daylight environments. Visual hierarchy and signal-type color coding is preserved across themes.



Light mode dashboard — same data, daylight styling.

Filters & Closed State

Users can filter by signal type or toggle "Show Closed" to review completed items. Each card has a **Mark as Done** button that syncs status to Google Sheets in real time via a dedicated webhook endpoint. A **Mark All Done** action with a 10-second undo banner enables bulk review.



Closed signals shown with strikethrough — status persists across sessions.

7. Limitations & Out of Scope

A clear-eyed inventory of what ThreadKeep **cannot** do today. These are deliberate MVP exclusions or known constraints — surfacing them upfront builds product credibility.

Functional Limitations

- **No proactive WhatsApp reading** ThreadKeep cannot watch all WhatsApp groups in real time. Users must explicitly forward messages to capture WhatsApp signals (by design — preserves privacy & ToS compliance).
- **No voice note transcription yet** Voice notes — heavily used in India — are not yet processed. Whisper integration is planned for Phase 2.

· **No image/screenshot OCR** Forwarded screenshots of conversations are not parsed yet. OCR fallback ingestion is on the roadmap.

· **No conflict detection** If the same person is assigned two tasks with overlapping deadlines across channels, ThreadKeep does not flag it. Planned for Phase 3.

· **No team-wide visibility** MVP is single-user only. Team dashboards, manager digests, and cross-member task views are explicitly post-MVP.

· **No automated reminders** Smart deadline reminders (24h before / 1h after) are designed but not yet shipped. Currently, surfacing depends on user-initiated review or SimSim query.

· **No native mobile app** Dashboard is web-only. A PWA wrapper or native app is post-launch.

Technical Limitations

· **Google Sheets as DB** Sheets is the MVP database. It will not scale beyond ~10K rows per user. Migration to Supabase (Postgres) is mapped for Phase 4.

· **GREEN-API dependency** WhatsApp ingestion currently relies on GREEN-API's unofficial WhatsApp Web bridge. This is non-compliant with Meta ToS at scale and will need replacement with official Meta Business API + dedicated number for production.

· **No multi-tenant architecture** The current setup is single-instance. Multi-user/multi-tenant isolation requires re-architecture before public launch.

· **Webhook reliability** No retry logic if Claude API or Google Sheets is down — failed extractions are dropped, not queued.

· **No PII redaction** Email content and chat text flow through Claude as-is. Production will require redaction of sensitive fields (account numbers, OTPs, etc.) before transmission.

Explicit Out of Scope (MVP)

· Replying to original WhatsApp/Gmail/Slack messages on the user's behalf

· Calendar integration (Google Calendar, Outlook)

· CRM integration (Salesforce, HubSpot)

· Auto-creating Jira/Linear/Asana tickets from extracted signals

· Sentiment or tone analysis on conversations

· Meeting transcription (handled well by Fireflies, Otter)

8. Execution & Trade-Offs

PM-Led Decisions

· **De-prioritised** Weekly digests and escalation queries from MVP scope to keep the build focused on the core extraction → storage → surface loop.

· **Chose Google Sheets over Postgres** as the MVP database to ship in days instead of weeks. Defined a clean migration path to Supabase for scale.

· **Pivoted from official Meta API to GREEN-API** when business verification and payment gates blocked progress — protecting velocity without sacrificing the vision.

· **Added duplicate detection later** rather than upfront, to avoid premature optimisation when the dual-Gmail issue was an edge case during early use.

· **Anchored MVP on accuracy over coverage** as the founding principle. False positives erode trust faster than missing signals — every feature was tested against this rule.

9. Impact & Outcomes

ThreadKeep shipped a working end-to-end MVP across four ingestion channels in a single sprint.

EXTRACTION ACCURACY	RETRIEVAL TIME	CHANNELS LIVE	SIGNAL TYPES
~95%	<30s	4	6
Validated on Hinglish + English	vs 15+ min previously	Gmail x2, Slack, WhatsApp	Tasks, Decisions, etc.

UPTIME	WHATSAPP FRICTION	CODE WRITTEN	SPRINT
24/7	<2s	0	1
Cloud-deployed on Railway	Per forward action	Built via no-code stack	End-to-end MVP delivery

Qualitative Outcomes

· **Defensible moat established** — only product simultaneously monitoring personal WhatsApp + Gmail + Slack with AI extraction.

· **Zero behaviour change** for Gmail and Slack ingestion; minimal incremental friction for WhatsApp via forwarding model.

· **Persistent state architecture** — closed signals survive page refresh, browser change, and laptop restart via Google Sheets sync.

· **Multi-language ready** — Hinglish phrases like *"kal tak bhej dena"* parse correctly without rule-based engineering.

· **Production-ready deployment** — full pipeline running 24/7 on Railway with proper credential isolation and webhook security.

10. Scale Roadmap — If Productionised

A staged plan for taking ThreadKeep from solo MVP to a production-grade B2B SaaS product. Each phase is gated by a measurable outcome and tied to specific scaling milestones.

Phase 1 — Closed Beta (Q3 2026)

Goal: Validate forwarding behaviour and extraction quality with 10 paid beta users from network.
Build: Voice note transcription (Whisper), promotional email filter, smart reminder system.
Migrate: WhatsApp ingestion from GREEN-API to official Meta Business API with dedicated number.
Gate: 70% Week-2 retention · ≥10 forwards/user/week.

Phase 2 — Public Launch (Q4 2026)

Goal: 100 paying users via Product Hunt India launch.
Build: Multi-tenant architecture, weekly digest emails, escalation snapshots, user authentication.
Migrate: Google Sheets to Supabase (Postgres). Add PII redaction layer before Claude calls.
Gate: 40% Week-4 retention · NPS ≥ 40.

Phase 3 — Team Tier (Q1 2027)

Goal: Land 10 paying teams (5+ seats each) via direct B2B outreach.
Build: Linear-based shared team dashboard, manager digest, conflict detection, team onboarding.
Add: Image/screenshot OCR ingestion. Conversation memory across signals.
Gate: ≥5 active teams · ARR > ■25L.

Phase 4 — Enterprise (Q2 2027)

Goal: First enterprise contract (■10L+/year).
Build: Self-hosted option, SSO, India data residency, audit logs, custom integration API.
Compliance: Begin SOC 2 Type 1 certification track. DPDP Act compliance review.
Gate: 1 enterprise logo signed · 99.9% uptime SLA achievable.

What Gets Hard at Scale

· **Privacy & compliance** Once paying users send their WhatsApp data, DPDP Act and SOC 2 obligations kick in. PII redaction, encryption at rest, and audit logs must precede growth — not follow it.

· **Cost economics of Claude API** At ~■0.50 per signal extraction, 1000 active users sending 50 signals/week costs ~■2L/month in API alone. Need batching, caching, and a cheaper fallback model for low-confidence cases.

· **WhatsApp official API constraints** Meta charges per business-initiated conversation. Reminder DMs from ThreadKeep would incur fees — pricing model must absorb or pass through this cost.

· **Hinglish & regional language depth** Current accuracy is good for North Indian Hinglish. Tamil/Telugu/Bengali code-switching needs separate prompt tuning & potentially fine-tuned models.

· **Multi-tenant data isolation** Sheets-based MVP has no isolation. A leak between two enterprise customers would be existential — Postgres RLS or per-tenant database becomes mandatory.

11. Success Metrics & OKRs

North Star Metric

Messages forwarded to ThreadKeep per active user per week. This single metric indicates the product is genuinely useful. If users trust ThreadKeep enough to forward their personal WhatsApp messages to it, adoption is real. For Gmail and Slack — where ingestion is automatic — the equivalent metric is *tasks auto-extracted per user per week*.

Quarter 1 OKRs (Post-Launch)

Objective	Key Result	Target
	Task extraction accuracy (user-validated)	≥ 90%
Users trust extraction	% of extracted tasks with correct assignee	≥ 85%
	Messages forwarded per active user per week	≥ 10
Users engage weekly	Weekly Active Users	≥ 70% of registered
	Weekly digest open rate	≥ 60%
Reduces escalation pain	"Easier to find past decisions" (NPS)	≥ 80% agree
	Avg time to resolve "who committed to what"	< 30 seconds
Retention signals PMF	Week-4 retention	≥ 40%
	Organic referrals per active user (Month 1)	≥ 2

12. Reflections as a Product Manager

The most important call wasn't a feature decision — it was rejecting the assumption that *the right solution must be the most technically elegant one*. Every competitor was waiting for personal WhatsApp to expose an API. By accepting the constraint and designing a forwarding model around it, ThreadKeep unlocked a solvable version of an "unsolvable" problem.

The discipline of **principle-first prioritisation** — "accuracy over coverage", "near-zero behaviour change", "background-first" — prevented scope creep at every fork. Every feature proposal was tested against these principles. If it conflicted, it was de-scoped without debate. This is what allowed a solo, no-code build to ship in a fraction of typical timelines.

If I were to do this again, I would invest earlier in **duplicate detection** (an edge case that surfaced during testing) and would have **filtered promotional emails upstream** rather than processing them through Claude — small cost-of-quality issues that compound at scale.

Built with intention. Shipped with discipline.
ThreadKeep is what happens when product thinking meets technical pragmatism.